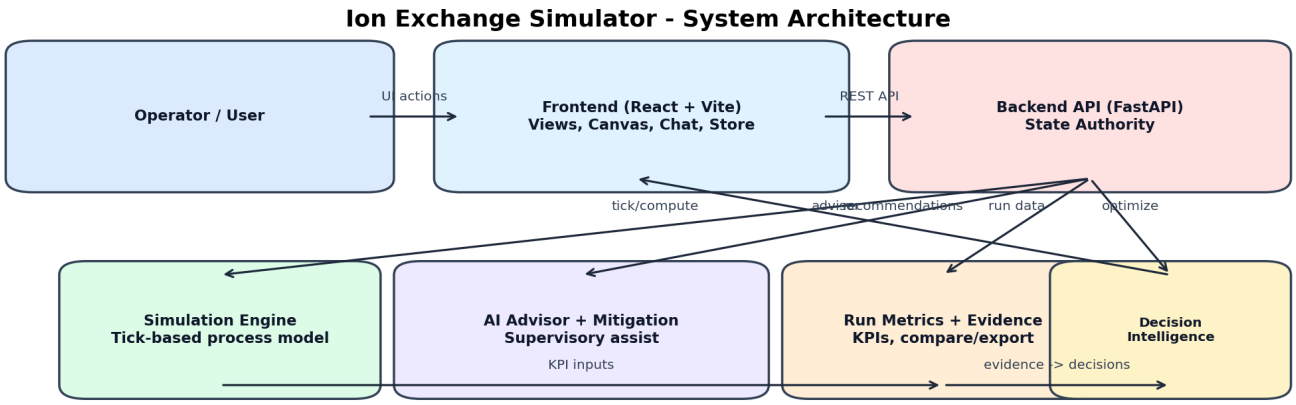
Ion Exchange Resin Simulator - Full System Documentation and User Manual

Comprehensive guide for new users and technical reviewers with backend logic detail and architecture diagrams.

Document Scope

- New-user onboarding, navigation, and operation procedures.
- Frontend-backend interaction model and runtime control loop.
- Backend endpoint responsibilities and process-engine behavior.
- Run evidence and decision-intelligence workflow.
- Troubleshooting playbooks and operational best practices.



1. System Overview

The simulator represents an ion-exchange resin process line with dual upstream synthesis paths (A/B), shared downstream constraints, and supervisory tooling for monitoring, mitigation, and decision support. The frontend is an operator console; the backend is state authority and process engine.

1.1 Core Software Components

Layer	Main Responsibility	Key Artifacts
Frontend	Operator interaction and visualization	React views, canvas, chat, run/decision screens
State Store	Client-side UI/process state	Zustand simulation store and polling control
Backend API	State sync and command handling	FastAPI endpoints under /state, /simulate, /runs, /decisions
Simulation Core	Tick-based process updates	Physics/proxy updates, alerts, inventory, history
Decision Layer	Ranking and recommendation	Run ranking, optimization candidates, explainability trace

1.2 Topology and Process Blocks

- Feed tanks supply styrene and DVB into two reactor lines.
- Reactor A and B each feed washer stages, then surge buffers.
- Both buffers converge into a shared dryer, then packaging/QC.
- Shared dryer acts as a natural capacity coordination point.

2. New User Guide

2.1 First-Time Startup Checklist

- Open frontend URL (typically localhost:5174).
- Confirm header does not show 'Backend Offline'.
- Verify recipe badge appears in top header (DVB/profile summary).
- Open Plant Twin view before initiating first batch.

2.2 First Batch Walkthrough

- Click Initiate Batch and observe tick progression and stage transitions.
- Monitor reactor temperature and conversion trends early (first 10-20 ticks).
- Watch washer->buffer->dryer handoff for congestion behavior.
- If critical alert appears, review advisor mitigation before resuming.
- End run to save KPI evidence for comparisons.

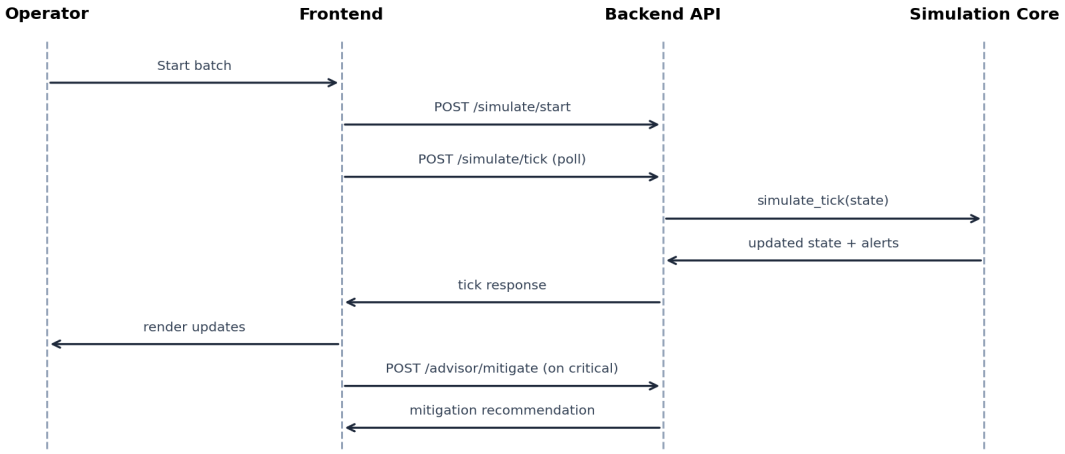
2.3 Primary Views and Their Purpose

View	What You Use It For
Live Dashboard	High-level operational status and trends
Plant Twin (Designer)	Real-time node-level process observation and control context
Production Logs	Chronological event and operation trace
Alert Matrix	Risk/event concentration by time and node
Inventory Pulse	Material availability and depletion monitoring
AI Advisor	Guidance, mitigation prompts, contextual analysis
Runs & Evidence	Experiment tracking, KPI evidence, compare/export
Decision Intelligence	Run ranking and recipe recommendations

3. Runtime Logic and Control Loop

3.1 Polling-Driven Tick Lifecycle

When simulation is enabled, frontend polling repeatedly calls POST /simulate/tick. Backend computes the next state and returns updated nodes, edges, alerts, history information, recipe snapshot, and selected aggregate metrics (energy, bottleneck IDs).



Runtime tick loop, sync path, and safety interlock

3.2 Safety Interlock Behavior

- Frontend checks tick response for critical error alerts.
- On critical alert, simulation is auto-paused and chat panel is opened.
- Frontend requests mitigation recommendation from advisor endpoint.
- Operator can apply bounded mitigation actions (e.g., cooling/rpm/inventory actions).

4. Backend Logic Deep Dive

4.1 Backend Domain Responsibilities

- State endpoints: hydrate/sync/reset master process state.
- Simulation endpoints: start/stop/tick and scenario injections.
- Advisor endpoints: chat and mitigation recommendation workflows.
- Run endpoints: start/end/abort/list/compare/export evidence handling.
- Decision endpoints: run ranking, optimize, recommend-next, explain.

4.2 Simulation Engine Mechanics

- Per-tick stage update and node traversal with type-specific logic.
- Reactor logic uses conversion progression, exotherm/cooling, quality proxies, safety trip checks.
- Storage and inventory logic tracks consumption and stockout/reorder conditions.
- Buffer and dryer logic mediates flow continuity and downstream constraints.
- History points and aggregate metrics are updated for dashboard and KPI use.

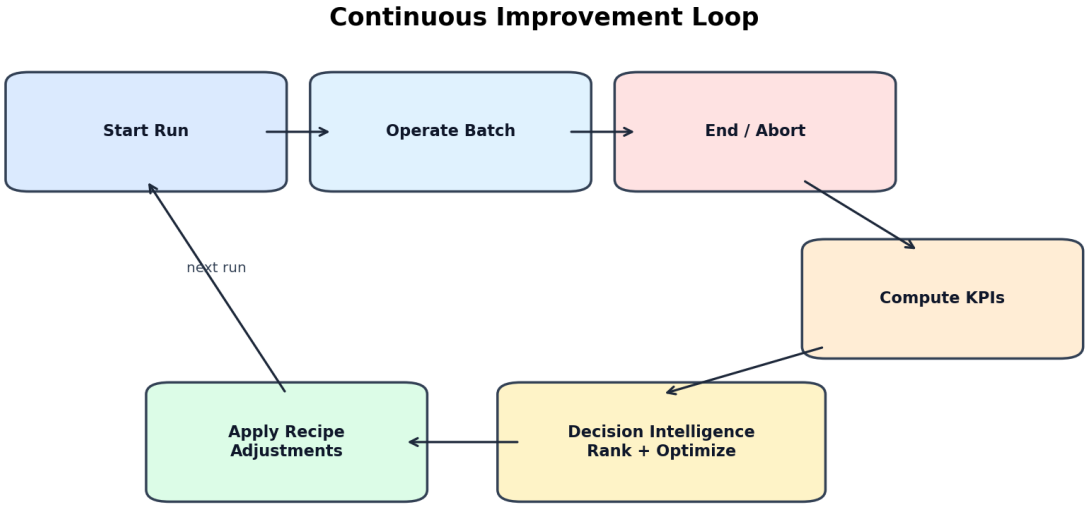
4.3 Run Metrics and Comparative Evidence

- Run metrics aggregate terminal state + run history slice.
- KPIs include thermal, conversion, quality proxy, energy, and alert counters.
- Run comparison computes delta and short narrative between two runs.
- Run evidence becomes the basis for decision-intelligence ranking and optimization.

4.4 Decision Intelligence Pipeline

- Rank completed/aborted runs using weighted KPI scoring.
- Select baseline run/recipe, then generate constrained candidate recipe grid.
- Predict KPI proxies per candidate and compute weighted candidate score.
- Return top-N candidates with confidence and explainability trace.
- Explain endpoint returns candidate trace for a specific run context.

5. Continuous Improvement Workflow



Operational Pattern

- Run baseline and capture clean evidence.
- Run a controlled variation with one recipe change at a time.
- Compare KPI deltas and inspect risk/quality tradeoffs.
- Use decision candidates to define next safe experiment.
- Repeat with evidence discipline to avoid blind tuning.

Recommended Governance for Team Use

- Always label runs with scenario intent.
- Never compare runs with missing/aborted context without annotation.
- Record mitigation events during unstable runs.
- Treat decision outputs as advisory unless externally calibrated.

6. Troubleshooting and FAQ

6.1 Common Issues

- 404 on /decisions/*: wrong backend instance/port running.
- CORS-like browser errors: backend 500 exceptions often appear as CORS failures in browser console.
- Frontend not reflecting env change: restart Vite after .env.local edits.
- Multiple port listeners: stale backend processes can keep old versions active.

6.2 Fast Recovery Checklist

- Verify backend health endpoint responds.
- Verify frontend API base URL points to active backend port.
- Check openapi routes include expected decision endpoints.
- Restart frontend after backend/env changes.

6.3 Practical Limitation Statement

Current process and decision mechanisms are advanced simulation heuristics/proxies and strong for training, demonstrations, and internal decision support loops. For production-grade plant control authority, additional calibration, validation, and governance are required.